

Redmine

Importador masivo de imputaciones

Arquitectura, flujo de datos e integración con la API REST de Redmine mediante un backend Express y un frontal React.

Índice

1. Qué es esta aplicación

El problema real que se quería resolver y para quién.

2. Por qué un backend Express y no solo un frontal

Las cinco razones técnicas que obligan a tener un servidor por medio.

3. Arquitectura general

Las tres piezas del sistema y cómo se hablan entre ellas.

4. Estructura del repositorio

Qué hay en cada carpeta y por qué está donde está.

5. El flujo completo de una importación

Desde el clic en «Importar» hasta la entrada creada en Redmine.

6. La API del backend

Los diez endpoints, con sus parámetros y respuestas.

7. Llamadas contra la API de Redmine

Todo lo que el servidor pide o envía a Redmine.

8. La plantilla Excel

Cómo se genera, qué validaciones lleva y cómo se lee de vuelta.

9. La validación fila a fila

El orden exacto de comprobaciones y los mensajes de error.

10. Credenciales y seguridad

Dónde vive la API key y por qué el servidor no la guarda.

11. Despliegue

Desarrollo, producción de un solo proceso y contenedor.

12. Limitaciones conocidas

Lo que hoy no hace y conviene tener presente.

1. Qué es esta aplicación

Imputar el parte de horas del mes en Redmine, una a una, por la interfaz web, es lento y repetitivo. Esta aplicación permite rellenar un Excel y volcarlo entero de golpe.

Redmine registra el trabajo en forma de *time entries*: cada una es una fecha, unas horas, un proyecto o una tarea, un tipo de actividad y un comentario. Su interfaz web está pensada para meterlas de una en una, lo que a fin de mes se traduce en decenas de formularios idénticos. La API REST de Redmine sí permite crearlas de forma programática, pero exige una API key y hablar JSON, algo que no está al alcance de cualquiera.

Esta herramienta cubre ese hueco con dos funciones concretas:

- **Importar** — descargas una plantilla Excel ya rellena con tus proyectos y actividades reales, la completas con las imputaciones del mes y la subes. La aplicación las va creando en Redmine una a una y te enseña el resultado de cada fila en tiempo real.
- **Consultar** — eliges un rango de fechas y ves lo que ya tienes imputado en ese periodo, con el total de horas, para cuadrar antes de cerrar el mes.

El punto de partida es que cada persona usa **su propia API key**. La aplicación no tiene usuarios, ni base de datos, ni sesiones: es un intermediario que traduce un Excel a llamadas contra la API de Redmine, usando siempre las credenciales de quien está delante de la pantalla. Los permisos, por tanto, son exactamente los que esa persona ya tiene en Redmine — ni uno más.

1.1 A quién va dirigida

A cualquier miembro del equipo que impute horas en Redmine. No requiere conocimientos técnicos más allá de saber dónde encontrar la propia API key (*Mi cuenta > Clave de acceso API*) y manejar un Excel. Está pensada para desplegarse una sola vez en una máquina accesible por la red interna o la VPN, y que el resto la use desde el navegador.

2. Por qué un backend Express y no solo un frontal

Es la pregunta razonable: si Redmine ya expone una API REST y el navegador sabe hacer peticiones HTTP, ¿por qué no atacarla directamente desde React y ahorrarse el servidor? La respuesta es que hay cinco cosas que el navegador, solo, no puede hacer. Cada una por separado ya justificaría el backend; juntas no dejan alternativa.

2.1 CORS: Redmine no autoriza a tu navegador

Un navegador solo permite que una página en `http://miapp:3001` llame a `https://redmine.empresa.com` si ese servidor devuelve explícitamente cabeceras `Access-Control-Allow-Origin` que lo autoricen. Redmine, de serie, no las envía. La petición ni siquiera llegaría a salir: el navegador la bloquea antes.

Esa política es del navegador, no del protocolo. Un proceso Node no la aplica: hace la petición HTTP y punto. Al colocar Express por medio, el frontal solo habla con su propio origen — que sí se autoriza a sí mismo — y es el servidor quien habla con Redmine. El problema desaparece por construcción.

2.2 La API key no debe pasarse por el navegador más de lo imprescindible

Una API key de Redmine equivale a la contraseña de esa persona a efectos de API: permite leer y escribir todo lo que ella puede. Si el frontal atacase Redmine directamente, esa clave viajaría en cada petición desde el navegador y quedaría expuesta a cualquier extensión o script de la página.

Aquí la clave viaja en dos cabeceras (`X-Redmine-Base-Url` y `X-Redmine-API-Key`) desde el navegador hasta *nuestro* servidor, y de ahí a Redmine. El backend **no la persiste en ningún sitio**: la recibe en la petición, la usa y la olvida. Sin base de datos, sin fichero de configuración, sin log.

DECISIÓN DE DISEÑO

Que el servidor no guarde credenciales tiene una consecuencia buena y una mala. La buena: un despliegue comprometido no filtra las claves de nadie, porque no hay nada que filtrar. La mala: cada usuario tiene que introducir la suya en Ajustes, y solo se recuerda en el `localStorage` de su propio navegador si marca la casilla.

2.3 Generar y leer Excel es trabajo de servidor

La plantilla no es un fichero estático: se construye al vuelo con los proyectos y actividades reales de quien la pide, y lleva dentro validaciones de datos, desplegados y fórmulas `VLOOKUP`. La biblioteca que hace esto (`ExcelJS`) funciona en Node con soltura; llevarla al navegador significaría cargar cientos de kilobytes de JavaScript para hacer allí lo que el servidor hace en milisegundos.

Y a la vuelta ocurre lo mismo: el `.xlsx` que sube el usuario hay que abrirlo, localizar la hoja «Datos», recorrer las filas y resolver el valor real de cada celda — que puede ser texto, un número, una fecha o el resultado de una fórmula. Es un parseo con suficientes casos raros como para no querer hacerlo en el cliente.

2.4 Una importación no es una petición: es un trabajo largo

Subir cincuenta filas significa cincuenta llamadas `POST` contra Redmine, cada una con su latencia. Eso no cabe en el ciclo petición-respuesta clásico: el navegador se quedaría minutos con la rueda girando, sin saber si va por la fila 3 o la 47, y a merced del primer *timeout* del camino.

El backend lo resuelve partiendo la operación en dos. Primero valida y responde `202 Accepted` con un identificador de trabajo, en menos de un segundo. Después, cuando el frontal se engancha al *stream*, procesa las filas y va emitiendo eventos SSE (*Server-Sent Events*) según avanza. El usuario ve la barra moverse y cada fila resolverse en directo. Sin backend no hay dónde alojar ese trabajo ni desde dónde emitir esos eventos.

2.5 Redmine paginado y con certificados imperfectos

La API de Redmine devuelve como mucho 100 elementos por página. Sacar la lista completa de proyectos son N llamadas encadenadas leyendo `total_count` y desplazando el `offset`; lo mismo para consultar las imputaciones de un rango. Ese bucle vive mucho mejor en el servidor, que además puede reutilizar el resultado.

Aparte está el asunto de los certificados. Muchos Redmine internos sirven una cadena TLS incompleta — les falta el certificado intermedio. Los navegadores lo toleran porque completan la cadena por su cuenta; Node no, y falla con `UNABLE_TO_VERIFY_LEAF_SIGNATURE`. Tener un servidor propio permite darle una salida a ese caso (la variable `REDMINE_ALLOW_INSECURE_TLS`, acotada solo a las llamadas a Redmine) sin tocar nada más.

2.6 De propina: un solo proceso en producción

Como ya existe el servidor Express, en producción se le encarga también servir el frontal ya compilado. Si `client/dist` existe, Express lo publica como estático y redirige cualquier ruta que no empiece por `/api` al `index.html`. Resultado: **un único proceso Node y un único puerto** para toda la aplicación. Nada de dos servidores, ni proxy inverso, ni CORS entre partes propias.

```
const clientDist = path.join(__dirname, '../..client/dist');
const hasClientBuild = fs.existsSync(clientDist);

if (hasClientBuild) {
  app.use(express.static(clientDist));
  app.get(/^(?!\/api).*/, (_req, res) => {
    res.sendFile(path.join(clientDist, 'index.html'));
  });
}
```

En desarrollo esa carpeta no existe, así que el bloque no hace nada y es Vite quien sirve el frontal en su propio puerto con recarga en caliente. El mismo código vale para los dos escenarios sin condicionales por entorno.

3. Arquitectura general

Tres piezas, dos saltos de red. El navegador nunca habla con Redmine; Redmine nunca sabe que existe un frontal. Todo pasa por el backend, que actúa de traductor: recibe Excel y devuelve eventos, recibe JSON y devuelve JSON.

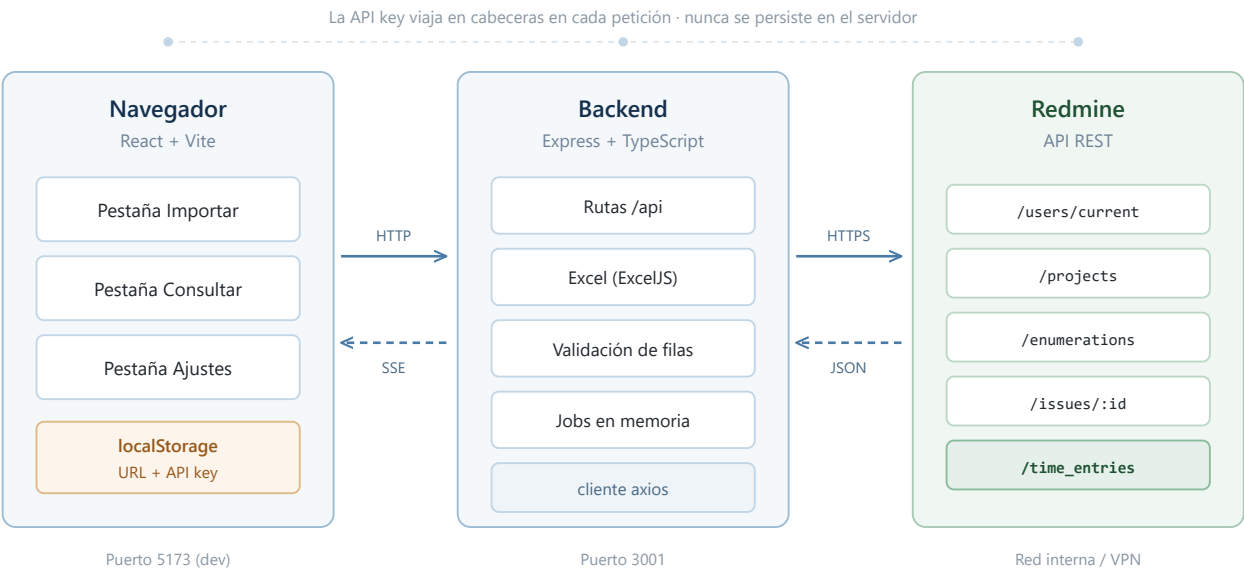


Figura 1. Las tres piezas del sistema. En producción, el frontal lo sirve el propio Express y el puerto 5173 desaparece.

3.1 Stack técnico

Capa	Tecnología	Papel que cumple
Frontal	React 18 + TypeScript, Vite	Tres pestañas, estado en <i>contexts</i> , sin librería de estado externa.
Backend	Express + TypeScript	Rutas <code>/api</code> , proxy hacia Redmine y servidor de estáticos en producción.
Cliente HTTP	axios	Instancia creada por llamada con la URL base y la API key de esa petición.
Excel	ExcelJS	Genera la plantilla con validaciones y parsea el fichero subido.
Subida de ficheros	multer (<i>memory storage</i>)	El <code>.xlsx</code> nunca toca el disco: se procesa desde memoria.
Tiempo real	SSE nativo + EventEmitter	Progreso fila a fila sin WebSockets ni dependencias añadidas.
Monorepo	npm workspaces	<code>server</code> y <code>client</code> como paquetes; un solo <code>npm install</code> .

SIN BASE DE DATOS

No hay persistencia de ningún tipo. Los trabajos de importación viven en un `Map` en memoria con diez minutos de vida, y la única fuente de verdad es Redmine. Reiniciar el proceso no pierde nada que importe: como mucho, una importación a medias, que se puede repetir.

4. Estructura del repositorio

Monorepo con dos *workspaces*. La regla que se ha seguido es que cada carpeta del servidor tenga una responsabilidad clara y que las rutas no contengan lógica de negocio: se limitan a orquestar.

```
server/src/ |— index.ts Arranque: puerto, host, dotenv |— app.ts Montaje de Express, rutas y
estáticos |— types.ts Tipos compartidos del dominio |— routes/ Un fichero por grupo de endpoints |
|— meta.routes.ts Conexión, proyectos, actividades, tareas | |— template.routes.ts Descarga de la
plantilla | |— import.routes.ts Subida, stream SSE y consulta de job | |— timeEntries.routes.ts
Consulta por rango de fechas |— middleware/ | |— redmineCreds.ts Extrae y valida las cabeceras de
credenciales | |— errorHandler.ts Traduce errores de red/TLS a mensajes legibles |— redmine/ Todo
lo que habla con la API de Redmine | |— client.ts Instancia axios + traducción de errores | |—
projects.ts Listado paginado de proyectos | |— activities.ts Tipos de actividad | |— issues.ts
Consulta de una tarea concreta | |— timeEntries.ts Crear y listar imputaciones |— excel/ | |—
templateBuilder.ts Construye el .xlsx con validaciones | |— importParser.ts Lee el .xlsx subido |—
import/ | |— rowValidator.ts Reglas de validación de cada fila | |— jobStore.ts Map en memoria con
TTL de 10 min | |— jobProcessor.ts Bucle de proceso + emisión de eventos |— utils/ |— dates.ts
Normalización de fechas y rangos |— sse.ts Cabeceras y formato de eventos SSE client/src/ |—
App.tsx Layout y navegación por pestañas |— pages/ ImportTab · ConsultaTab · SettingsTab |— api/
Envoltorio de fetch + un fichero por área |— context/ Credenciales · Favoritos · Proyectos de
confianza |— hooks/ useImportStream: consumo del SSE |— components/ ProgressBar · tablas de
resultados
```

La separación entre `redmine/` e `import/` es deliberada: la primera carpeta sabe hablar con Redmine pero no sabe nada de Excel ni de trabajos; la segunda coordina el proceso pero delega toda llamada externa. Si algún día cambia la API de Redmine, el cambio se queda contenido en `redmine/`.

5. El flujo completo de una importación

Este es el recorrido que hace un dato desde que se teclea en una celda de Excel hasta que existe como imputación en Redmine. Es el corazón de la aplicación y conviene entenderlo entero, porque explica por qué la respuesta llega en dos tiempos.

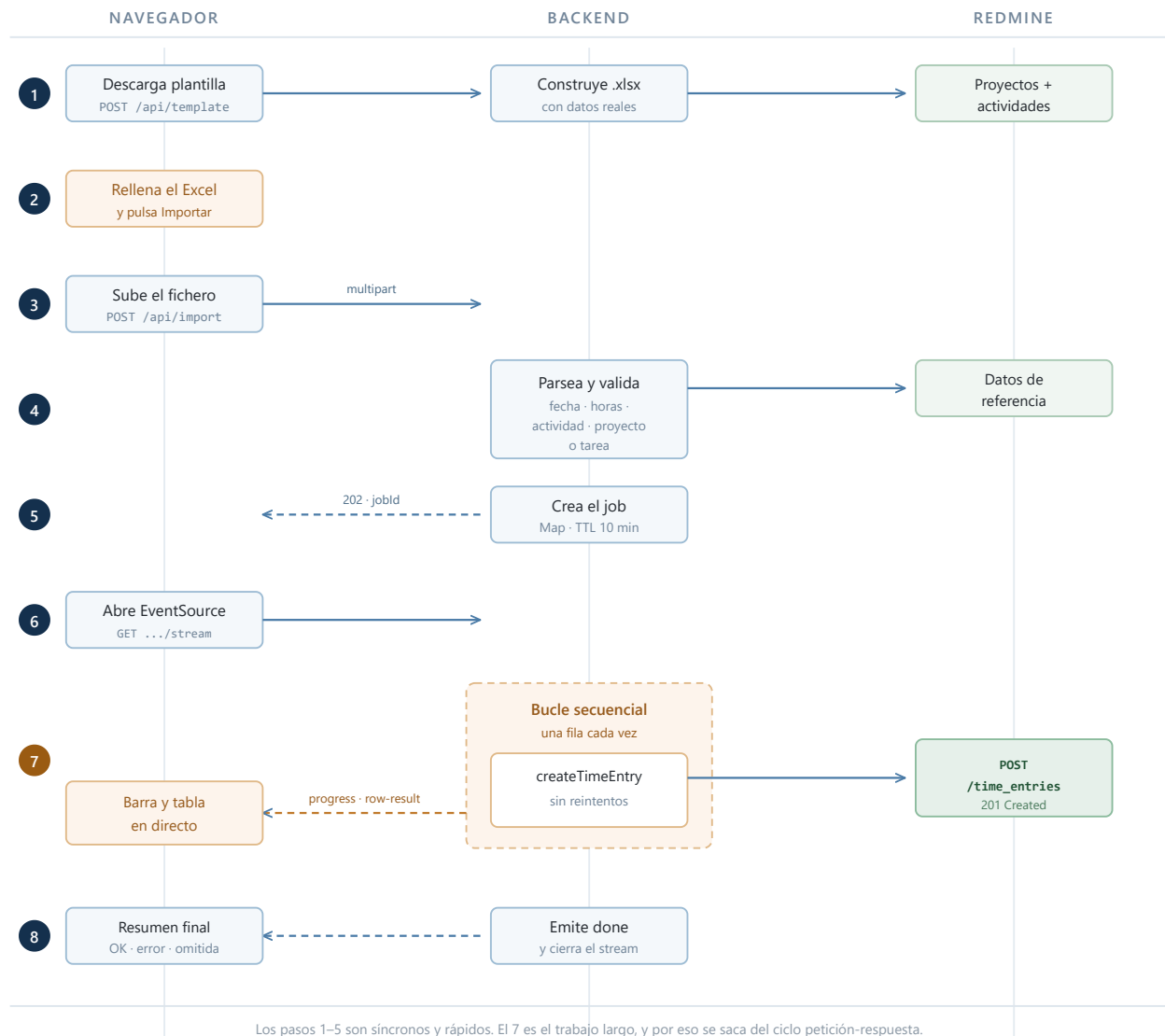


Figura 2. Ciclo completo de una importación. La respuesta llega en dos tiempos: primero el `jobId`, luego los resultados por SSE.

5.1 Los pasos, uno a uno

Paso 1 — Descargar la plantilla

El usuario pulsa «Descargar plantilla». El frontal envía un `POST /api/template` con la lista de tareas frecuentes que tenga guardadas. El servidor pide a Redmine *en paralelo* los proyectos y los tipos de actividad, y con eso construye un `.xlsx` a medida. La plantilla no es genérica: lleva los desplegables ya cargados con los proyectos reales de esa persona.

Paso 2 — Rellenar el Excel

Aquí no interviene la aplicación. El usuario completa las filas en su Excel, aprovechando los despleables y las fórmulas que trae la plantilla. Es importante entender que **los datos no se teclean en un formulario HTML**: las celdas del Excel *son* el formulario.

Paso 3 — Subir el fichero

Al pulsar «Importar», el frontal manda un `POST /api/import` como `multipart/form-data` con tres cosas: el fichero, las tareas frecuentes y los ajustes de proyectos de confianza. Las credenciales van, como siempre, en las cabeceras.

Paso 4 — Parsear y validar

El servidor abre el libro en memoria, localiza la hoja «Datos» y recorre las filas mapeando las siete columnas. Descarta las vacías y marca como omitidas las que ya llevan «Sí» en la columna *Enviado*. Con los proyectos y actividades recién traídos de Redmine, valida el resto una a una. Nada se ha enviado todavía a Redmine.

Paso 5 — Crear el trabajo y responder

Las filas válidas y los errores de validación se guardan en un objeto `Job` con un UUID, en un `Map` en memoria con diez minutos de vida. El servidor responde `202 Accepted` con `{ jobId, totalRows }`. Este es el momento clave: **la petición HTTP termina aquí**, sin haber creado ni una sola imputación.

Paso 6 — Engancharse al stream

Con el `jobId` en la mano, el frontal abre un `EventSource` contra `GET /api/import/:jobId/stream`. El servidor responde con cabeceras de *event-stream*, reenvía de golpe los resultados que ya tuviera (los errores de validación del paso 4) y solo entonces arranca el procesamiento.

POR QUÉ EL PROCESO ARRANCA AQUÍ Y NO ANTES

El trabajo empieza cuando alguien se conecta al *stream*, no al subir el fichero. Así se garantiza que ningún resultado se pierda por haberse emitido antes de que hubiera quien lo escuchara. Un `Set` de trabajos ya iniciados evita que dos conexiones simultáneas lo lancen dos veces.

Paso 7 — Procesar fila a fila

El bucle recorre las filas válidas **en secuencia**, esperando a que cada `POST /time_entries.json` termine antes de lanzar el siguiente. Por cada una emite dos eventos: `progress` con el contador y `row-result` con el desenlace. Si Redmine rechaza una fila, se anota el motivo y **se sigue con la siguiente**: un fallo no aborta la importación.

Paso 8 — Cerrar

Al terminar, se emite `done` con el recuento de éxitos, fallos y omitidas, y se cierra la conexión. El frontal deja en pantalla la tabla completa para que el usuario pueda revisar qué falló y corregirlo en el Excel.

6. La API del backend

Diez endpoints, todos bajo `/api`. Salvo `/api/health`, todos exigen las cabeceras de credenciales; si faltan, la respuesta es `400` antes de tocar nada.

6.1 Cabeceras comunes

Cabecera	Contenido
<code>X-Redmine-Base-Url</code>	URL base del Redmine, p. ej. <code>https://redmine.dominio.com</code> . Se valida como URL antes de usarla.
<code>X-Redmine-API-Key</code>	API key personal del usuario (<i>Mi cuenta</i> > <i>Clave de acceso API</i> en Redmine).

6.2 Diagnóstico y datos de referencia

Verbo	Ruta	Qué hace y qué devuelve
<code>GET</code>	<code>/api/health</code>	Comprueba que el proceso vive. Devuelve <code>{ ok: true }</code> . Es lo que usa el <code>HEALTHCHECK</code> del contenedor.
<code>POST</code>	<code>/api/meta/test-connection</code>	Valida las credenciales llamando a <code>/users/current.json</code> . Devuelve <code>{ ok, userName }</code> con el nombre real, para que el usuario confirme que es su cuenta. Un <code>401</code> se propaga como <code>401</code> ; el resto de fallos como <code>502</code> .
<code>GET</code>	<code>/api/meta/projects</code>	Lista completa de proyectos visibles, resolviendo la paginación de Redmine por dentro. Devuelve <code>{ projects: [{ id, name, identifier }] }</code> .
<code>GET</code>	<code>/api/meta/activities</code>	Tipos de actividad configurados en la instancia. Devuelve <code>{ activities: [{ id, name }] }</code> .
<code>GET</code>	<code>/api/meta/issues/:id</code>	Datos de una tarea concreta: <code>{ issue: { id, subject, projectId, projectName } }</code> . Se usa para resolver a qué proyecto pertenece una tarea. <code>400</code> si el id no es numérico, <code>404</code> si no existe.

6.3 Plantilla

Verbo	Ruta	Qué hace y qué devuelve
<code>POST</code>	<code>/api/template</code>	Genera el <code>.xlsx</code> al vuelo. Recibe <code>{ favorites: [...] }</code> en el cuerpo JSON y devuelve el binario con <code>Content-Disposition: attachment</code> y nombre <code>plantilla_imputaciones.xlsx</code> . Es <code>POST</code> y no <code>GET</code> porque lleva cuerpo: la lista de tareas frecuentes que se incrusta en la hoja de referencia.

6.4 Importación

Verbo	Ruta	Qué hace y qué devuelve
POST	/api/import	Recibe <code>multipart/form-data</code> con <code>file</code> (el .xlsx), <code>favorites</code> y <code>trustedProjects</code> (ambos JSON serializado). Parsea, valida y crea el trabajo. Responde 202 con <code>{ jobId, totalRows }</code> . Devuelve 400 si no llega fichero o si la plantilla no tiene filas de datos.
GET	/api/import/:jobId/stream	Abre el <i>stream</i> SSE y arranca el proceso. Emite <code>progress</code> , <code>row-result</code> y <code>done</code> . Reenvía primero lo ya acumulado, así que reconectar no pierde resultados. 404 si el trabajo no existe o caducó.
GET	/api/import/:jobId	Estado del trabajo en formato JSON, como alternativa al <i>stream</i> : <code>{ status, processed, total, results }</code> . Si ya terminó, la respuesta lo borra de memoria. 404 si no existe.

Los tres eventos del stream

Evento	Carga útil
progress	<code>{ processed, total }</code> — alimenta la barra.
row-result	El desenlace de una fila. Éxito: <code>{ rowIndex, status: 'success', spentOn, project, hours, timeEntryId }</code> . Fallo u omisión: <code>{ rowIndex, status: 'error' 'skipped', reason }</code> .
done	<code>{ processed, total, succeeded, failed, skipped }</code> — resumen y cierre.

6.5 Consulta

Verbo	Ruta	Qué hace y qué devuelve
GET	/api/time-entries?from&to	Imputaciones propias entre dos fechas (YYYY-MM-DD). Devuelve <code>{ entries, totalHours }</code> , resolviendo la paginación por dentro. 400 si el formato es incorrecto o <code>from</code> es posterior a <code>to</code> .

6.6 Tratamiento de errores

Un *middleware* final traduce los fallos de red y TLS a mensajes en castellano, porque un `ENOTFOUND` pelado no le dice nada a quien solo quiere imputar sus horas:

Código de Node	Mensaje que ve el usuario
<code>ENOTFOUND</code> · <code>EAI_AGAIN</code>	No se pudo resolver la URL de Redmine. Revisa la URL y el acceso de red.
<code>ECONNREFUSED</code>	Redmine rechazó la conexión. Revisa que el servicio esté accesible.
<code>ETIMEDOUT</code> · <code>ECONNABORTED</code>	La conexión ha tardado demasiado. Revisa la red o la VPN.
<code>UNABLE_TO_VERIFY_LEAF_SIGNATURE</code> y afines	No se pudo verificar el certificado TLS (cadena incompleta). Remite al README.

7. Llamadas contra la API de Redmine

Todo lo que sale hacia Redmine pasa por una instancia de axios creada en `redmine/client.ts`, con la API key en la cabecera `X-Redmine-API-Key`, veinte segundos de `timeout` y `validateStatus: () => true` — es decir, ningún código HTTP lanza excepción: cada función decide qué hacer con el estado que recibe.

Verbo	Endpoint de Redmine	Cuándo y para qué
GET	<code>/users/current.json</code>	Al pulsar «Probar conexión». Valida la clave y recupera el nombre del usuario.
GET	<code>/projects.json</code>	Al generar la plantilla y al importar. Paginado de 100 en 100 hasta agotar <code>total_count</code> .
GET	<code>/enumerations/time_entry_activities.json</code>	Junto al anterior, en paralelo. Da la lista de actividades válidas.
GET	<code>/issues/:id.json</code>	Solo si los proyectos de confianza están activos: averigua a qué proyecto pertenece realmente una tarea. Con caché por importación.
POST	<code>/time_entries.json</code>	La llamada que escribe. Una por fila válida. <code>201</code> es éxito.
GET	<code>/time_entries.json</code>	Pestaña Consultar. Filtra por <code>user_id=me</code> y <code>spent_on=<desde hasta></code> , paginando.

7.1 El cuerpo que se envía a Redmine

Cada fila válida se traduce a este JSON. La disyuntiva importante está al final: una imputación va contra **una tarea o contra un proyecto, nunca contra ambos**. Si la fila trae número de tarea, se manda `issue_id` y Redmine deduce el proyecto solo.

```
POST /time_entries.json
X-Redmine-API-Key: <la key del usuario>
Content-Type: application/json

{
  "time_entry": {
    "spent_on": "2026-07-14",
    "hours": 7.5,
    "activity_id": 9,
    "comments": "Revisión del importador",
    "issue_id": 12345
  }
}
```

La respuesta `201` trae el identificador de la entrada creada, que se guarda en el resultado de la fila y acaba mostrándose en la tabla. Cualquier otro código se traduce a un motivo legible: si Redmine devuelve un array `errors`, se concatena tal cual — suele ser el mensaje más útil, porque viene del propio Redmine y explica exactamente qué rechazó.

7.2 El caso del certificado

Si el Redmine interno sirve una cadena TLS incompleta, Node se niega a conectar. La solución correcta es que el servidor sirva la cadena completa, o arrancar con `NODE_EXTRA_CA_CERTS` apuntando a la CA interna. Como atajo para redes controladas existe la variable `REDMINE_ALLOW_INSECURE_TLS=true`.

CUIDADO CON ESTO

`REDMINE_ALLOW_INSECURE_TLS` desactiva la verificación del certificado, lo que abre la puerta a un ataque de intermediario. Su alcance está acotado a las llamadas a Redmine y a nada más, pero no debe usarse si el tráfico atraviesa una red que no controlas.

8. La plantilla Excel

La plantilla es la pieza que hace usable todo lo demás. No es un fichero que alguien dejó en una carpeta: se genera en cada descarga con los datos reales de quien la pide, y lleva dentro suficiente inteligencia como para que sea difícil equivocarse.

8.1 Dos hojas

Hoja «Datos» — donde se escribe

Col	Cabecera	Contenido y validación
A	Fecha	Validación de tipo fecha, acotada entre 2020 y 2035.
B	Proyecto	Desplegable con los proyectos reales. Se autorrellena por fórmula si se elige una tarea frecuente.
C	Nº Tarea	Un número de tarea, o el nombre de una tarea frecuente (desplegable).
D	Actividad	Desplegable con las actividades reales. Obligatorio.
E	Horas	Número mayor que cero. Se acepta coma o punto decimal.
F	Comentario	Texto libre. Opcional.
G	Enviado	Desplegable «Sí / No». Si vale «Sí», la fila se omite.

Hoja «Referencia» — el catálogo

Contiene los proyectos (columna A), las actividades (columna B) y la tabla de tareas frecuentes (columnas D a G: etiqueta, nº de tarea, proyecto y actividad). Es la fuente de los desplegables y de las fórmulas. Sirve además como chuleta: si un nombre no aparece aquí, no se va a validar.

8.2 El truco de las tareas frecuentes

Si el usuario tiene tareas frecuentes configuradas, la plantilla incrusta en las columnas B y D una fórmula que las autorrellena al elegir una tarea del desplegable de la columna C:

```
B{fila} = IFERROR(VLOOKUP($C{fila}, Referencia!$D$2:$G$n, 3, FALSE), "")
D{fila} = IFERROR(VLOOKUP($C{fila}, Referencia!$D$2:$G$n, 4, FALSE), "")
```

El efecto para quien lo usa es que elige «Soporte diario» en la columna de tarea y el proyecto y la actividad aparecen solos. Es un detalle pequeño que ahorra la mayor parte de los errores de tecleo. La preparación se aplica a las primeras 500 filas, que es el límite práctico de la plantilla.

8.3 La vuelta: leer el fichero

Al subir, el parser tiene que lidiar con que una celda de Excel no siempre contiene lo que parece. Puede ser un `Date`, un objeto con `formula` y su `result`, un objeto con `text` para texto enriquecido, o un valor plano. La función de lectura los resuelve todos, y para las fórmulas se queda con el resultado calculado — que es justo lo que se ve en pantalla.

Las fechas se normalizan a `YYYY-MM-DD` aceptando tres formatos: ISO directo, `dd/mm/yyyy` (el habitual en Excel con configuración regional española) y, como último recurso, lo que el constructor `Date` de JavaScript sea capaz de interpretar. Las filas completamente vacías se descartan sin ruido.

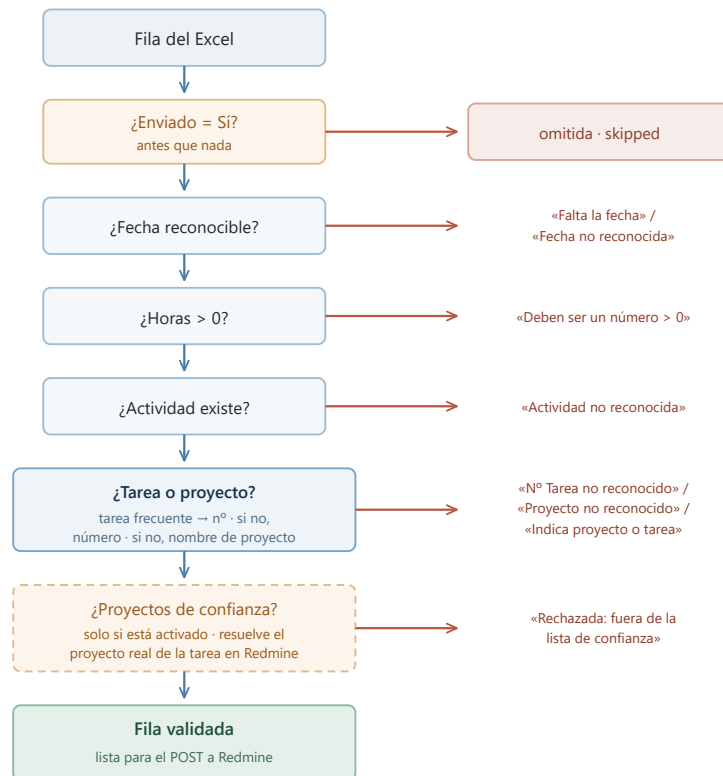
Para «Enviado» se acepta `sí`, `si`, `true`, `x` o `yes`, sin distinguir mayúsculas y quitando los acentos antes de comparar — de ahí que «Sí» y «si» funcionen igual.

PARA QUÉ SIRVE REALMENTE LA COLUMNA «ENVIADO»

Es la red de seguridad contra los duplicados. Si una importación falla a medias, marcas como «Sí» las filas que sí entraron y vuelves a subir el mismo fichero: las omitidas no se reintentan. Sin ella, cada reintento duplicaría lo ya imputado, porque Redmine no tiene forma de saber que esa entrada ya existe.

9. La validación fila a fila

Antes de enviar nada, cada fila pasa por una cadena de comprobaciones en orden fijo. La primera que falla corta y produce el mensaje de error: no se acumulan varios motivos por fila, se da el primero, que suele ser el que hay que arreglar.



La primera comprobación que falla corta la cadena y produce el mensaje. Los nombres se comparan sin distinguir mayúsculas ni espacios sobrantes.

Figura 3. Cadena de validación. Todo esto ocurre *antes* de que se envíe nada a Redmine.

9.1 Cómo se resuelve el destino de la imputación

Es la parte con más matiz. El campo «Nº Tarea» admite dos cosas distintas y se resuelve en este orden:

1. Se busca el texto entre las **tareas frecuentes** del usuario. Si coincide con una etiqueta, se usa su `issueId`.
2. Si no, se intenta interpretar como **número**. Si lo es, ese es el id de la tarea.
3. Si no es ninguna de las dos, la fila se rechaza.
4. Si la columna venía vacía, se cae al **proyecto**: se busca por nombre y se usa su id. Si tampoco hay proyecto, error.

Todas las comparaciones de nombres son insensibles a mayúsculas y a espacios sobrantes, porque copiar y pegar desde Redmine arrastra ambas cosas con frecuencia.

9.2 Lo que no se valida por adelantado

La validación previa comprueba lo que se puede saber sin escribir: formatos, y que proyectos y actividades existan. No comprueba si *esa* persona tiene permiso para imputar en *ese* proyecto, si la tarea está cerrada, o si hay reglas de negocio propias de la instancia. Eso lo dirá Redmine al recibir el `POST`, y su mensaje de error se muestra tal cual en la tabla de resultados — de ahí que se propaguen literalmente los `errors` que devuelve.

10. Credenciales y seguridad

10.1 El recorrido de la API key

La clave se introduce en la pestaña Ajustes y vive en el estado de React. Si el usuario marca «Recordar», se guarda en el `localStorage` de su navegador bajo la clave `redmineCreds`; si no, desaparece al recargar. En cada petición se adjunta en la cabecera `X-Redmine-API-Key`. El servidor la lee, la mete en `req.redmine` para el resto de la petición, la usa para construir el cliente axios y la descarta al terminar.

Durante una importación hay una excepción: la clave se copia dentro del objeto `Job`, porque el proceso sigue vivo después de que la petición original haya terminado y necesita seguir autenticándose contra Redmine. Ese objeto vive en memoria y se destruye a los diez minutos, o antes si se consulta el resultado final.

10.2 Lo que esto implica

Propiedad	Consecuencia práctica
Sin persistencia en servidor	Comprometer el despliegue no filtra las claves de nadie: no hay base de datos ni fichero donde estén.
Sin sesiones ni usuarios	La aplicación no tiene identidad propia. Los permisos son los que cada API key ya tiene en Redmine.
Clave en <code>localStorage</code>	Accesible por JavaScript del mismo origen. Es el compromiso aceptado a cambio de no reintroducirla cada vez.
Clave en memoria durante el job	Ventana de diez minutos como máximo. Un volcado de memoria en ese lapso la expondría.
CORS abierto	<code>app.use(cors())</code> sin restricción de origen. Aceptable en red interna; conviene acotarlo si el despliegue se abre más.

A TENER EN CUENTA AL DESPLEGAR

El tráfico entre el navegador y el backend va en HTTP plano. Dentro de una VPN que ya cifra el túnel el riesgo añadido es bajo, pero la API key viaja en claro por esa red. Si eso preocupa, la salida es un proxy inverso (Caddy o Nginx) con certificado delante del proceso Node.

10.3 Proyectos de confianza

Es una defensa contra el error humano, no contra un ataque. Se configura una lista blanca de proyectos y, con la opción activa, cualquier fila que apunte fuera de ella se rechaza antes de enviarse. La lista por defecto viene precargada en el frontal y se guarda en `localStorage`.

La parte interesante es que, cuando la fila trae un número de tarea, no basta con mirar la columna «Proyecto» del Excel: hay que preguntarle a Redmine a qué proyecto pertenece esa tarea *de verdad*, porque el Excel podría decir otra cosa. Eso es una llamada extra por tarea distinta, cacheada durante la

importación para no repetirla. Si no se puede resolver el proyecto real, la fila se rechaza: ante la duda, no se escribe.

11. Despliegue

11.1 Desarrollo

```
npm install
npm run dev
```

Levanta el backend en `http://localhost:3001` y Vite en `http://localhost:5173`, con recarga en caliente. Vite arranca con `--host`, así que también responde por la IP de la máquina en la red local.

11.2 Producción: un proceso

```
npm install
npm run build    # genera server/dist y client/dist
npm start        # node server/dist/index.js
```

Al existir `client/dist`, Express lo sirve y toda la aplicación queda tras el puerto 3001, escuchando en `0.0.0.0`. Variables reconocidas: `PORT`, `HOST` y `REDMINE_ALLOW_INSECURE_TLS`, léidas de `server/.env`.

Para que sobreviva a reinicios, pm2 o una tarea programada de Windows. Y hay que abrir el puerto en el cortafuegos, o los demás no llegarán:

```
New-NetFirewallRule -DisplayName "Redmine Import Activities" `
  -Direction Inbound -LocalPort 3001 -Protocol TCP `
  -Action Allow -Profile Private,Domain
```

11.3 Contenedor

El `Dockerfile` usa tres etapas para que la imagen final no arrastre nada de compilación: una compila con todas las dependencias, otra instala solo las de producción del *workspace* del servidor, y la última copia únicamente lo compilado sobre `node:20-alpine`. Expone el 3001 y trae un `HEALTHCHECK` contra `/api/health`.

En Kubernetes, el despliegue vive en su propio *namespace* y la imagen se descarga de un registry privado, lo que exige un *secret* de tipo `dockerconfigjson` en ese mismo *namespace* — los secrets no cruzan de uno a otro. Para publicar una versión nueva, el procedimiento está en `upgrade-version-readme.md`, en la raíz del repositorio.

SOBRE LAS ETIQUETAS DE IMAGEN

El despliegue usa `imagePullPolicy: IfNotPresent`. Con esa política, reutilizar la etiqueta `latest` no funciona: el nodo se queda con la copia que ya tiene en caché y nunca baja la nueva. Hay que usar etiquetas de versión (`v1.0.1`, `v1.0.2` ...) y actualizar el deployment con `kubectl set image`.

12. Limitaciones conocidas

Todo esto son decisiones conscientes de un proyecto pequeño, no descuidos. Conviene tenerlas presentes antes de estirar la herramienta más allá de para lo que se hizo.

Limitación	Detalle y por qué está así
Los jobs viven en memoria	Un reinicio del proceso pierde las importaciones en curso. Con un solo proceso y trabajos de segundos, montar Redis sería desproporcionado. Impide, eso sí, escalar a varias réplicas: el <i>stream</i> tiene que caer en el mismo proceso que tiene el job.
Proceso secuencial	Las filas se envían una a una. Cien filas son cien viajes de ida y vuelta. Paralelizar sería fácil, pero castigaría a Redmine y complicaría el orden de los resultados; para el volumen habitual no compensa.
Sin reintentos	Un fallo puntual de red marca la fila como error y sigue. La recuperación es manual: marcar «Enviado» en las que sí entraron y volver a subir.
Sin protección contra duplicados	Subir dos veces el mismo fichero sin usar la columna «Enviado» duplica las imputaciones. No hay clave de idempotencia porque Redmine no ofrece uno.
Límite de 500 filas	Es hasta donde llegan los desplegables y las fórmulas de la plantilla. El parser leería más, pero esas filas irían sin ayuda de validación.
Sin tests automatizados	No hay suite. Los candidatos naturales serían el validador de filas y el normalizador de fechas, que son lógica pura y fácil de cubrir.
CORS sin restringir	<code>cors()</code> abierto. Correcto en red interna, revisable si el despliegue se expone más.

12.1 Por dónde crecería

- **Marcar «Enviado» automáticamente** — devolver el Excel con la columna ya rellena según el resultado eliminaría el paso manual más molesto y, de paso, casi todo el riesgo de duplicados.
- **Reintento con espera progresiva** — un par de intentos ante errores de red (no ante rechazos de Redmine, que son deterministas) evitaría la mayoría de fallos transitorios.
- **Tests del validador** — es lógica pura, sin dependencias externas: la mejor relación entre esfuerzo y red de seguridad.
- **Caché de proyectos y actividades** — hoy se piden a Redmine en cada plantilla y cada importación. Cambian poco; unos minutos de caché ahorrarían varias llamadas.

EN UNA LÍNEA

La aplicación hace una cosa y la hace entera: convierte un Excel en imputaciones de Redmine, enseñando por el camino qué entró y qué no. El backend existe porque sin él nada de eso es posible desde un navegador — ni por CORS, ni por la clave, ni por el Excel, ni por el trabajo largo, ni por la paginación.